

La Table Commune

*MongoDB Restaurant Management System
Cloud Database Design and Implementation*

2026

Ista Kpovenon

Project Summary

La Table Commune is a MongoDB-based restaurant management system designed to demonstrate the implementation of a modern NoSQL database solution for the restaurant industry. The project covers the complete database development lifecycle, from data modeling and implementation to optimization, validation, security, and reporting.

The solution manages key business entities, including customers, branches, employees, menu items, ingredients, orders, reservations, deliveries, suppliers, and promotional campaigns. The database was implemented using MongoDB Atlas and follows best practices for document-oriented database design.

The project includes data import scripts, indexing strategies, aggregation pipelines for business analytics, schema validation using JSON Schema, role-based access control (RBAC) recommendations, reporting views, backup and recovery procedures, and query performance analysis using MongoDB's `explain()` functionality. Query optimization was achieved through targeted indexing and execution plan analysis.

Overall, the project demonstrates practical skills in MongoDB database design, administration, performance optimization, and analytical reporting while following industry best practices for scalable and maintainable NoSQL solutions.

SOMMAIRE

1. Introduction
 - 1.1 Contexte du projet
 - 1.2 Objectifs
 - 1.3 Présentation du système « La Table Commune »
2. Analyse et conception de la base de données
 - 2.1 Modélisation Entité-Association (E/A)
 - 2.2 Entités et attributs
 - 2.3 Diagramme ERD
 - 2.4 Relations et cardinalités
3. Conception NoSQL sous MongoDB
 - 3.1 Analyse des charges et choix des modèles de stockage
 - 3.2 Design Patterns MongoDB
 - 3.3 Structure des collections
 - 3.3 Exemples de documents
4. Implémentation
 - 4.1 Création de la base de données
 - 4.2 Création des collections
 - 4.3 Insertion des données
 - 4.4 Scripts MongoDB (mongosh)
5. Optimisation des performances
 - 5.1 Création des index
 - 5.2 Analyse des performances avec explain()
 - 5.3 Résultats avant et après indexation
6. Analyse des données
 - 6.1 Pipelines d'agrégation
 - 6.2 Analyse des ventes
 - 6.3 Jointures avec \$lookup
 - 6.4 Résultats des agrégations
7. Architecture distribuée
 - 7.1 Réplication (Replica Set)
 - 7.2 Configuration du Replica Set
 - 7.3 Sharding

7.4 Choix de la clé de sharding

8. Évolution du schéma et sécurité

8.1 Évolution du schéma

8.2 Ajout du programme de fidélité

8.3 Validation avec JSON Schema

8.4 Gestion des utilisateurs et des rôles (RBAC)

8.5 Audit et mesures de sécurité

9. Déploiement Cloud avec MongoDB Atlas

9.1 Création du cluster Atlas

9.2 Connexion avec MongoDB Compass

9.3 Déploiement de la base

9.4 Tests dans Atlas

11. Conclusion

12. Références

1. Introduction

1.1. Contexte du projet

Dans un contexte de restauration moderne, un restaurant ne se limite plus à servir des repas en salle. Il doit aujourd'hui gérer simultanément plusieurs activités, notamment la consommation sur place, les commandes à emporter, les livraisons, les réservations, les paiements électroniques et en espèces, le suivi des stocks, la gestion du personnel, la fidélisation de la clientèle ainsi que l'analyse des performances commerciales.

Le présent projet consiste à concevoir une base de données MongoDB pour une application de gestion de restaurants multiservice capable de répondre à ces besoins tout en offrant une architecture flexible, performante et évolutive.

Le système doit notamment permettre de gérer :

- plusieurs succursales ;
- plusieurs types de commandes (sur place, à emporter et en livraison) ;
- un menu dynamique et personnalisable ;
- la personnalisation des plats selon les préférences des clients ;
- la gestion des tables et des réservations ;
- la gestion des clients et des programmes de fidélité ;
- le suivi des paiements ;
- la livraison des commandes et les commentaires des clients ;
- la gestion des stocks d'ingrédients ;
- le suivi des employés, de leurs rôles et de leurs responsabilités.

L'objectif principal est de concevoir une base de données capable de supporter un environnement de restauration moderne en garantissant la cohérence des données, de bonnes performances de lecture et d'écriture, ainsi qu'une capacité d'évolution vers une architecture distribuée. Le choix de MongoDB s'appuie sur son modèle documentaire, sa flexibilité de schéma, ses mécanismes d'indexation, ses capacités d'agrégation et son support natif de la réplication et du sharding, qui en font une solution adaptée aux applications à forte croissance.

1.2. Objectifs

Le projet poursuit les deux objectifs principaux suivants :

Objectif 1 : Concevoir une base de données NoSQL adaptée à un système de restauration multiservice

Objectif 2 : Développer une solution performante, sécurisée et évolutive

1.3. Présentation du système « La Table Commune »

La Table Commune est une application de gestion de restaurants multiservice conçue pour centraliser l'ensemble des opérations liées à l'exploitation d'un réseau de restaurants.

Le système permet de gérer plusieurs succursales, chacune disposant de son propre personnel, de ses tables, de son menu et de son inventaire. Les clients peuvent consulter le menu, passer des commandes sur place, à emporter ou en livraison, effectuer des réservations, participer à un programme de fidélité et suivre l'historique de leurs commandes.

L'application assure également la gestion des paiements, des livraisons, des stocks d'ingrédients, des promotions et des employés selon leurs rôles et responsabilités.

L'architecture de la base de données repose sur MongoDB afin de tirer parti de son modèle documentaire, particulièrement adapté aux données semi-structurées, aux relations flexibles entre les entités et aux besoins d'évolution d'une application de restauration moderne. Cette approche permet d'obtenir une solution performante, évolutive et facilement extensible pour accompagner la croissance du système.

2. Analyse et conception de la base de données

2.1. Modélisation conceptuelle (Entité-Association)

La modélisation Entité-Association (E/A) constitue la base de la conception de la base de données. Elle permet d'identifier les principales entités, leurs attributs et leurs relations afin de représenter fidèlement le domaine métier.

Bien que MongoDB soit une base de données NoSQL, ce modèle reste essentiel. Alors qu'il conduit à des tables normalisées en SQL, il guide ici la conception des collections et des documents en s'appuyant sur l'imbrication (*embedding*) et le référencement (*referencing*).

2.1 Entités et Attributs

Le tableau suivant présente les 20 principales entités du système ainsi que leurs principaux attributs. Ces entités couvrent l'ensemble des fonctionnalités de l'application, notamment la gestion des clients, des commandes, des réservations, des paiements, des livraisons, des stocks, des employés et des succursales.

Tableau 1 : Tableau des entités

Entité	Attribut	Type MongoDB	Rôle	Description
GroupeRestaurant	_id	ObjectId	_id/PK	Identifiant unique auto-généré
	nomGroupe	string		Nom commercial du groupe
	siegeSocial	string		Adresse du siège social
	telephoneSiege	string		Téléphone principal
	emailSiege	string	UNIQUE	Courriel officiel unique
	siteWeb	string		URL du site web
	dateCreation	date		Date de fondation
	statut	string		Statut opérationnel
ConseilAdministration	conseilAdministration	array[object]	EMBEDDED	Liste membres CA (embedded)
	idCA	int		Identifiant du membre CA
	nom	string		Nom du membre
	prenom	string		Prénom du membre
	telephone	string		Téléphone direct
	nombreActions	int		Nombre d'actions détenues
	valeurAction	decimal		Valeur unitaire de l'action
	dateCreation	date		Date d'entrée au CA
Succursale	typeActionnaire	string		Type (fondateur, investisseur...)
	_id	ObjectId	_id/PK	Identifiant unique
	idGroupe	ObjectId	FK	Référence au groupe parent
	nomSuccursale	string		Nom de la succursale
	adresse	object	EMBEDDED	Adresse complète embedded
	telephone	string		Téléphone de la succursale
	email	string	UNIQUE	Courriel unique
	horaires	object	EMBEDDED	Horaires par jour

	statut	string		Statut opérationnel
Adresse	_id	int	_id	_idAdresse
	rue	string		Numéro et rue
	ville	string		Ville
	codePostal	string		Code postal
	province	string		Province/État
	pays	string		Pays
	appartement	int		Numéro d'appartement
Table	_id	ObjectId	_id/PK	Identifiant unique
	idSuccursale	ObjectId	FK	Référence à la succursale
	numeroTable	string		Code de la table
	capacite	int		Nombre de places
	zone	string		Zone dans le restaurant
	statutTable	string		Disponibilité temps réel
Client	_id	ObjectId	_id/PK	Identifiant unique
	nom	string		Nom de famille
	prenom	string		Prénom
	telephone	string		Numéro de téléphone
	email	string	UNIQUE	Courriel unique
	dateInscription	date		Date d'inscription
	pointsFidelite	int		Solde points fidélité
	preferences	array[string]		Préférences alimentaires
	adressePrincipale	object	EMBEDDED	Adresse embedded
Reservation	_id	ObjectId	_id/PK	Identifiant unique
	idClient	ObjectId	FK	Référence au client
	idSuccursale	ObjectId	FK	Référence à la succursale
	idTable	ObjectId	FK	Table attribuée
	dateReservation	date		Date de la réservation
	heureReservation	string		Heure prévue
	nbPersonnes	int		Nombre de convives
	statutReservation	string		État de la réservation
	commentaire	string		Note spéciale client
Commande	_id	ObjectId	_id/PK	Identifiant unique
	type	string	ISA	Discriminateur ISA
	idClient	ObjectId	FK	Référence au client
	idSuccursale	ObjectId	FK	Référence à la succursale

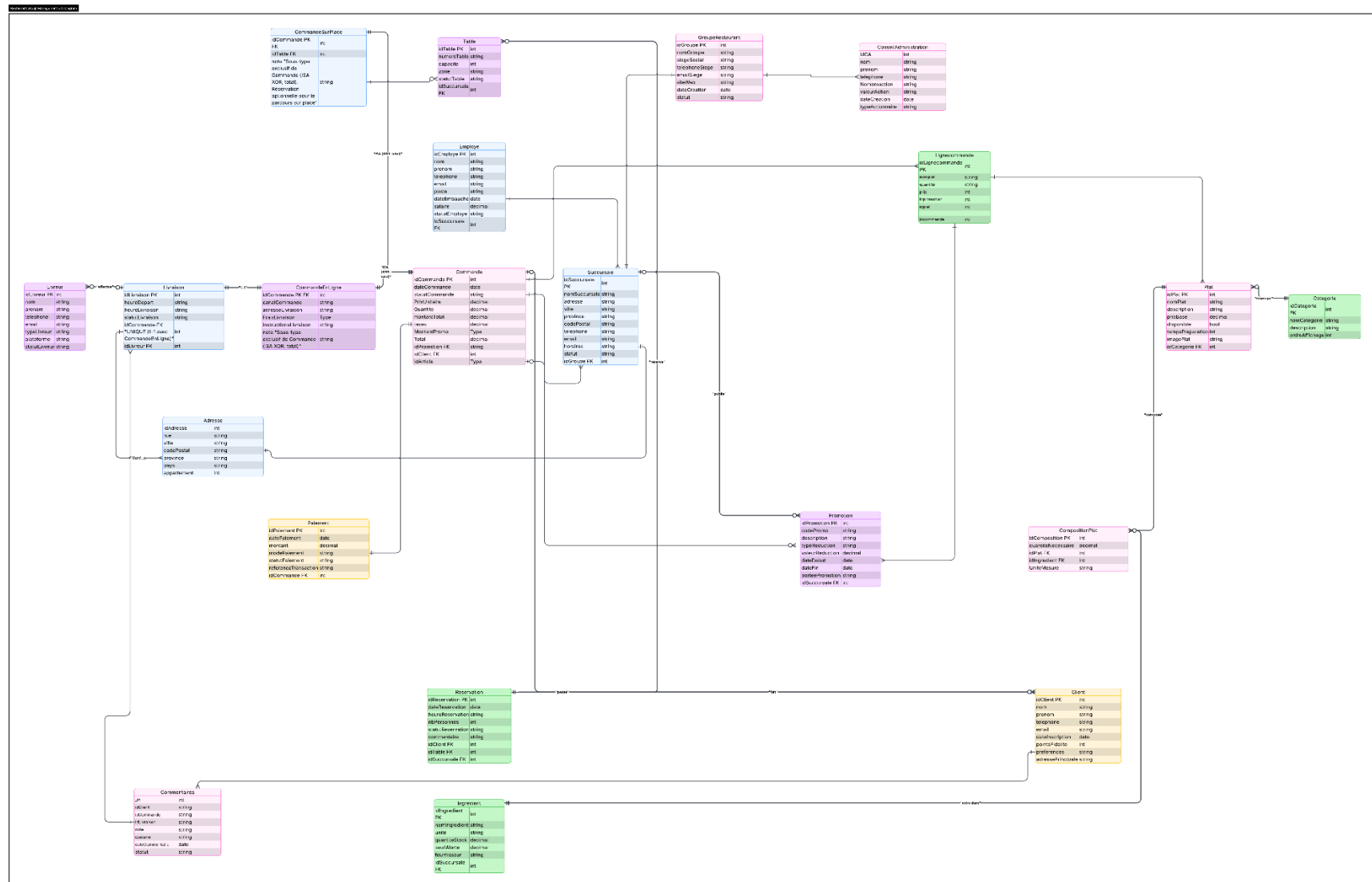
	idPromotion	ObjectId	FK	Promotion appliquée
	dateCommande	date		Horodatage commande
	statutCommande	string		État de la commande
	lignesCommande	array[object]	EMBEDDED	Détail plats commandés
	montantSousTotal	decimal		Sous-total avant promo/taxes
	montantPromo	decimal		Montant de la réduction
	taxes	decimal		Taxes calculées
	montantTotal	decimal		Total final payé
	paiement	object	EMBEDDED	Info paiement embedded
LigneCommande	idLignecommande	ObjectId	_id/PK	Identifiant unique
	nomPlat	string		Nom du plat
	Quantite	string		
	prix	string		
	idpromotion	int		Référence à une promotion appliquée
	idCommande	int		Référence à la commande à laquelle appartient la ligne
	idPlat	int		Référence au plat command
CommandeSurPlace	idCommande	ObjectId	_id/PK	Hérite de Commande (ISA XOR)
	idTable	ObjectId	FK	Table utilisée
	note	string		Note sur le parcours sur place
CommandeEnLigne	idCommande	ObjectId	_id/PK	Hérite de Commande (ISA XOR)
	canalCommande	string		Canal de commande
	adresseLivraison	object	EMBEDDED	Adresse livraison embedded
	fraisLivraison	decimal		Frais de livraison
	instructionsLivraison	string		Instructions livreur
	note	string		Note ISA sous-type
Livraison	_id	ObjectId	_id/PK	Identifiant unique
	idCommande	ObjectId	FK	Référence CommandeEnLigne (1:1)
	idLivreur	ObjectId	FK	Référence au livreur
	heureDepart	string		Heure de départ
	heureLivraison	string		Heure de livraison effective

	statutLivraison	string		Statut de livraison
Categorie	_id	ObjectId	_id/PK	Identifiant unique
	nomCategorie	string		Nom de la catégorie
	description	string		Description courte
	ordreAffichage	int		Ordre d'affichage dans le menu
Plat	_id	ObjectId	_id/PK	Identifiant unique
	idCategorie	ObjectId	FK	Référence à la catégorie
	nomPlat	string		Nom du plat
	description	string		Description pour le menu
	prixBase	decimal		Prix de base
	disponible	bool		Disponibilité temps réel
	tempsPreparation	int		Temps de préparation (min)
	imagePlat	string		URL image du plat
CompositionPlat	_id	ObjectId	_id/PK	Identifiant unique
	idPlat	ObjectId	FK	Référence au plat
	idIngredient	ObjectId	FK	Référence à l'ingrédient
	quantiteNecessaire	decimal		Quantité requise par portion
	uniteMesure	string		Unité de mesure
Ingredient	_id	ObjectId	_id/PK	Identifiant unique
	idSuccursale	ObjectId	FK	Référence à la succursale
	nomIngredient	string		Nom de l'ingrédient
	unite	string		Unité de mesure
	quantiteStock	decimal		Quantité en stock
	seuilAlerte	decimal		Seuil déclenchant alerte
	fournisseur	string		Nom du fournisseur
Employe	_id	ObjectId	_id/PK	Identifiant unique
	idSuccursale	ObjectId	FK	Référence à la succursale
	nom	string		Nom de famille
	prenom	string		Prénom
	telephone	string		Numéro de téléphone
	email	string	UNIQUE	Courriel professionnel unique
	poste	string		Rôle dans le restaurant
	dateEmbauche	date		Date de début d'emploi
	salaire	decimal		Taux horaire

	statutEmploye	string		Statut actif/inactif
Livreur	_id	ObjectId	_id/PK	Identifiant unique
	nom	string		Nom de famille
	prenom	string		Prénom
	telephone	string		Numéro de téléphone
	email	string		Courriel
	typeLivreur	string		Interne ou externe
	plateforme	string		Plateforme partenaire
	statutLivreur	string		Disponibilité
Promotion	_id	ObjectId	_id/PK	Identifiant unique
	idSuccursale	ObjectId	FK	Succursale concernée
	codePromo	string	UNIQUE	Code promotionnel unique
	description	string		Description de la promo
	typeReduction	string		Pourcentage ou montant fixe
	valeurReduction	decimal		Valeur de la réduction
	dateDebut	date		Début de validité
	dateFin	date		Fin de validité
	porteePromotion	string		Portée de la promotion
Paiement	idPaieement	ObjectId	PK	Identifiant unique
	idCommande	ObjectId	FK	Commande concernée
	datePaieement	date		Date/heure du paiement
	montant	decimal		Montant payé
	modePaieement	string		Mode de paiement
	statutPaieement	string		Statut de la transaction
	ReferenceTransaction	string		Reference de transaction
Commentaires	_id	ObjectId	Identifiant unique	Identifiant unique du commentaire
	idClient	ObjectId	Référence	Client ayant laissé le commentaire
	idCommande	ObjectId	Référence (optionnel)	Commande sur place associée au commentaire
	idLivraison	ObjectId	Référence (optionnel)	Livraison associée au commentaire
	note	Number	Évaluation	Note donnée par le client (ex: sur 5)
	contenu	String	Contenu métier	Texte du commentaire laissé par le client

Source : Élaboration de l'auteur.

2.2 Diagramme ERD



2.3 Relations et cardinalités dans le modèle

Tableau 2 : Relations et cardinalités

Entité A	Relation	Entité B	Cardinalité E/A	Cardinalité / modélisation MongoDB
GroupeRestaurant	regroupe	Succursale	1,N	1 groupe référencé par plusieurs succursales (Succursale.idGroupe)
GroupeRestaurant	possède	ConseilAdministration	1,N	1 document GroupeRestaurant contient un tableau embedded conseilAdministration[]
Succursale	possède	Table	1,N	1 succursale référencée par plusieurs tables (Table.idSuccursale)
Succursale	reçoit	Reservation	1,N	1 succursale référencée par plusieurs réservations (Reservation.idSuccursale)
Succursale	enregistre	Commande	1,N	1 succursale référencée par plusieurs commandes (Commande.idSuccursale)
Succursale	emploie	Employe	1,N	1 succursale référencée par plusieurs employés (Employe.idSuccursale)
Succursale	stocke	Ingredient	1,N	1 succursale référencée par plusieurs ingrédients (Ingredient.idSuccursale)
Succursale	propose	Promotion	1,N	1 succursale référencée par plusieurs promotions (Promotion.idSuccursale)
Client	effectue	Reservation	1,N	1 client référencé par plusieurs réservations (Reservation.idClient)
Client	passe	Commande	1,N	1 client référencé par plusieurs commandes (Commande.idClient)

Table	est attribuée à	Reservation	1,N	1 table référencée par plusieurs réservations dans le temps (Reservation.idTable)
Table	est utilisée par	CommandeSurPlace	1,N	1 table référencée par plusieurs commandes sur place dans le temps (CommandeSurPlace.idTable)
Commande	spécialisation ISA XOR totale	CommandeSurPlace	1,1 spécialisation	Sous-type séparé, même identifiant (idCommande), héritage par référence / collection fille
Commande	spécialisation ISA XOR totale	CommandeEnLigne	1,1 spécialisation	Sous-type séparé, même identifiant (idCommande), héritage par référence / collection fille
Commande	contient	lignesCommande	1,N	1 commande contient un tableau embedded lignesCommande[]
Commande	donne lieu à	Paielement	1,1 (selon diagramme)	Paielement est embedded dans Commande; donc 1 document commande → 1 sous-document paielement
CommandeEnLigne	est suivie par	Livraison	1,1	1 livraison référence 1 commande en ligne (Livraison.idCommande UNIQUE)
Livreur	effectue	Livraison	1,N	1 livreur référencé par plusieurs livraisons (Livraison.idLivreur)
Categorie	contient	Plat	1,N	1 catégorie référencée par plusieurs plats (Plat.idCategorie)
Plat	est composé de	CompositionPlat	1,N	1 plat référencé par plusieurs lignes de composition (CompositionPlat.idPlat)
Ingredient	entre dans	CompositionPlat	1,N	1 ingrédient référencé par plusieurs lignes de composition (CompositionPlat.idIngredient)

Plat	est en relation M,N avec	Ingredient	M,N	Réalisé via collection intermédiaire CompositionPlat
Promotion	est appliquée à	Commande	1,N	1 promotion peut être référencée par plusieurs commandes (Commande.idPromotion)
Client	possède	Adresse	1,1	adressePrincipale est un objet embedded dans Client
Succursale	possède	Adresse	1,1	Adresse est un objet embedded dans Succursale
CommandeEnLigne	est livrée à	Adresse	1,1	adresseLivraison est un objet embedded dans CommandeEnLigne

Source : Élaboration de l'auteur.

3. Conception NoSQL sous MongoDB

3.1. Analyse des charges et choix des modèles de stockage

L'analyse des charges consiste à identifier les principales opérations du système, leur fréquence d'exécution et leur importance. Elle permet d'orienter les choix de modélisation, notamment l'utilisation de l'imbrication (*embedding*), du référencement (*referencing*), ainsi que la définition des index pour optimiser les performances.

Tableau 3 : Analyse des charges

Opérations principales	Type	Données accédées	Fréquence	Priorité	Modélisation MongoDB
Consulter le menu par catégorie	Lecture	Categorie, Plat	Très élevée	Haute	Référencement simple (Plat → Categorie)
Vérifier disponibilité d'un plat	Lecture	Plat.disponible, Ingredient	Très élevée	Haute	Référencement + autre (via CompositionPlat)

Créer une commande sur place	Écriture	Commande, CommandeSurPlace, Client, Table, paiement	Très élevée	Critique	Mixte : Référencement + Imbrication objet (paiement) + Imbrication tableau (lignesCommande)
Créer une commande en ligne	Écriture	Commande, CommandeEnLigne, Client, adresseLivraison, paiement	Très élevée	Critique	Mixte : Référencement + Imbrication objet (adresseLivraison, paiement) + Imbrication tableau
Ajouter/modifier lignes commande	Écriture /MàJ	Commande.lignesCommande[]	Très élevée	Critique	Imbrication tableau
Consulter une commande	Lecture	Commande + sous-types	Très élevée	Haute	Mixte (embedded + références)
Mettre à jour statut commande	Mise à jour	Commande.statutCommande	Très élevée	Critique	Document unique (aucune relation)
Appliquer promotion	Lecture + MàJ	Promotion, Commande	Élevée	Haute	Référencement simple (idPromotion)
Enregistrer paiement	Mise à jour	Commande.paiement	Très élevée	Critique	Imbrication objet
Consulter paiements	Lecture	Commande.paiement	Moyenne	Moyenne	Imbrication objet
Créer réservation	Écriture	Reservation, Client, Table, Succursale	Élevée	Haute	Référencement simple
Vérifier disponibilité table	Lecture	Table, Reservation	Élevée	Haute	Référencement simple

Modifier/annuler réservation	Mise à jour	Reservation	Moyenne	Haute	Document simple
Associer table à commande	Lecture + MàJ	CommandeSurPlace, Table	Élevée	Haute	Référencement simple
Créer livraison	Écriture	Livraison, CommandeEnLigne, Livreur	Élevée	Haute	Référencement simple
Mettre à jour livraison	Mise à jour	Livraison	Élevée	Haute	Document simple
Suivi livraison	Lecture	Livraison, Livreur	Élevée	Haute	Référencement simple
Consulter stock	Lecture	Ingredient	Élevée	Haute	Référencement simple
Déduire stock	Mise à jour	Ingredient, CompositionPlat	Élevée	Critique	Autre (collection associative CompositionPlat)
Vérifier ingrédients plat	Lecture	CompositionPlat, Ingredient	Élevée	Haute	Autre (relation M:N via collection)
Alerte stock faible	Lecture	Ingredient.seuilAlerte	Moyenne	Haute	Document simple
Ajouter ingrédient	Écriture	Ingredient	Moyenne	Moyenn e	Document simple
Ajouter plat	Écriture	Plat, Categorie	Faible	Moyenn e	Référencement simple
Modifier disponibilité plat	Mise à jour	Plat.disponible	Élevée	Haute	Document simple
Gérer composition plat	Écriture /MàJ	CompositionPlat	Moyenne	Moyenn e	Autre (collection associative)
Inscrire client	Écriture	Client, adressePrincipale	Moyenne	Moyenn e	Imbrication objet
Consulter client	Lecture	Client, Commande	Moyenne	Moyenn e	Mixte (embedded + références)
Mettre à jour fidélité	Mise à jour	Client.pointsFidelite	Élevée	Haute	Document simple

Gérer promotions	Écriture /MàJ	Promotion, Succursale	Moyenne	Moyenn e	Référencement simple
Consulter promotions	Lecture	Promotion	Élevée	Haute	Référencement simple
Gérer employés	CRUD	Employe, Succursale	Faible	Moyenn e	Référencement simple
Gérer livreurs	CRUD	Livreur	Moyenne	Moyenn e	Document simple
Ventes par succursale	Lecture analytique	Commande, Paiement, Succursale	Moyenne	Haute	Référencement simple
Commandes d'un client	Lecture	Commande, Client	Moyenne	Moyenn e	Référencement simple
Livraisons par livreur	Lecture	Livraison, Livreur	Moyenne	Moyenn e	Référencement simple
Gérer succursales	CRUD	GroupeRestaurant, Succursale	Faible	Moyenn e	Référencement simple
Conseil d'administration	Mise à jour	GroupeRestaurant.con seilAdministration[]	Faible	Basse	Imbrication tableau
Stocker un commentaire	Ecriture	Commande, livraison, client	Élevée	élevé	

Source : Élaboration de l'auteur

- CRUD = 4 types d'opérations sur les données : C Create (Créer), R Read (Lire), U Update (Mettre à jour), D Delete (Supprimer)
- MàJ = Mise à jour

3.2. Design pattern MongoDB

Dans ce projet, la base de données MongoDB a été conçue à partir du diagramme Entité-Association et des besoins d'un système de gestion de restaurant multiservice. Les choix de modélisation ont été faits en tenant compte de la fréquence d'utilisation des données, de leur niveau de dépendance, ainsi que des cardinalités (peu, plusieurs, beaucoup) afin d'optimiser les performances.

Les choix de modélisation reposent sur les cardinalités :

- Peu (1 à 1) → imbrication objet
- Plusieurs (1 à N modéré) → imbrication tableau
- Beaucoup ou partagé → référencement
- Plusieurs à plusieurs → collection intermédiaire

Cette approche permet d'obtenir une base de données performante, cohérente et adaptée aux besoins du système.

Tableau 4 : Synthèse des pattern utilisés

Patterns utilisés	Explications	Exemples
Embedded Document Pattern (documents imbriqués)	Ce pattern est utilisé lorsque : - il y a peu de données associées, - ces données sont fortement dépendantes, - et elles sont toujours utilisées avec le parent	- paiement dans Commande - adressePrincipale dans Client - adresseLivraison dans CommandeEnLigne Ici, on a une relation de type 1 à 1 (peu), donc l'imbrication est idéale. Cela permet d'éviter les jointures et d'améliorer la rapidité.
Embedded Array Pattern (tableaux imbriqués)	Ce pattern est utilisé lorsque : - il y a plusieurs éléments liés à un même parent, - mais en nombre raisonnable (pas énormément), - et qu'ils sont souvent lus ensemble.	Exemples : - lignesCommande[] dans Commande - conseilAdministration[] dans GroupeRestaurant Ici, on est dans une cardinalité 1 à plusieurs. Tant que le nombre reste contrôlé, l'imbrication est performante.
Reference Pattern (référencement)	Le référencement est utilisé lorsque : - il y a beaucoup de données, - ou des données partagées entre plusieurs documents, - ou encore des relations plusieurs à plusieurs.	Exemples : - Commande → Client - Plat → Catégorie - Livraison → Livreur - Reservation → Table - Commentaires → commande, client, livraison Ici, on est dans des cardinalités :

		• 1 à plusieurs (beaucoup) ou • plusieurs à plusieurs Le référencement évite la duplication et permet de gérer les données indépendamment.
Polymorphic Pattern	Ce pattern est utilisé pour gérer plusieurs types d'une même entité.	Exemple : • CommandeSurPlace • CommandeEnLigne Chaque commande a une base commune, mais aussi des attributs spécifiques. Cela permet de gérer plusieurs variantes d'un même objet sans dupliquer la structure.
Computed Pattern	Ce pattern est utilisé lorsque certaines données sont calculées.	Exemples : • montantSousTotal • taxes • montantTotal Ces valeurs sont calculées puis stockées pour éviter de refaire les calculs. Utile surtout quand il y a beaucoup de lectures.
Collection intermédiaire (relation N:N)	Utilisée lorsque : • une relation implique beaucoup d'éléments des deux côtés.	Exemple : • Plat ↔ Ingredient via CompositionPlat Ici, on a une relation plusieurs à plusieurs (beaucoup ↔ beaucoup). Une collection intermédiaire est donc nécessaire.

Source : Élaboration de l'auteur.

3.3. Structure des collections

À l'issue de la phase de conception, le modèle logique est constitué de 15 collections principales, représentant les entités essentielles du système :

- groupeRestaurant
- succursale
- table
- client
- reservation
- commande (une seule collection avec type)
- livraison
- livreur
- categorie
- plat
- ingredient
- compositionPlat (relation N:N)
- promotion
- employe
- Commentaires

Certaines informations ne sont pas stockées dans des collections distinctes, mais directement imbriquées dans les documents afin d'optimiser les performances et de limiter les jointures :

- adresse (*embedded document*)
- paiement (*embedded document*)
- conseilAdministration (*embedded array*)
- lignesCommande (*embedded array*)

3.4. Exemples de documents

- **GroupeRestaurant**

```
{
  "_id": ObjectId(),
  "nomGroupe": "Groupe Saveurs",
  "siegeSocial": "Montréal",
  "telephoneSiege": "5140000000",
```

```

    "emailSiege": "contact@groupe.com",
    "siteWeb": "www.groupe.com",
    "dateCreation": ISODate("2020-01-01"),
    "statut": "actif",
    "conseilAdministration": [
      {
        "nom": "Jean Dupont",
        "role": "Président"
      }
    ]
  }
}

```

- **Succursale**

```

{
  "_id": ObjectId(),
  "nomSuccursale": "Succursale Aylmer",
  "adresse": {
    "rue": "123 rue Principale",
    "ville": "Gatineau",
    "province": "QC",
    "codePostal": "J9H1A1"
  },
  "telephone": "8190000000",
  "email": "aylmer@groupe.com",
  "horaires": {
    "lundi": "9h-21h"
  },
  "statut": "actif",
  "idGroupe": ObjectId()
}

```

- **Client**

```

{
  "_id": ObjectId(),
  "nom": "Ista",
  "prenom": "K",
  "telephone": "8191111111",
  "email": "ista@email.com",
  "dateInscription": ISODate(),
  "pointsFidelite": 120,
}

```

```

    "adressePrincipale": {
      "rue": "456 rue Exemple",
      "ville": "Gatineau"
    }
  }
}

```

- **Commande**

```

{
  "_id": ObjectId(),
  "idClient": ObjectId(),
  "idSuccursale": ObjectId(),
  "type": "en_ligne",
  "dateCommande": ISODate(),
  "statutCommande": "en_cours",

  "lignesCommande": [
    {
      "idPlat": ObjectId(),
      "nomPlat": "Pizza",
      "quantite": 2,
      "prixUnitaire": 15
    }
  ],

  "montantSousTotal": 30,
  "taxes": 4.5,
  "montantTotal": 34.5,

  "paiement": {
    "mode": "carte",
    "statut": "payé",
    "datePaiement": ISODate()
  },

  "adresseLivraison": {
    "rue": "456 rue Exemple",
    "ville": "Gatineau"
  }
}

```

- **Réservation**

```
{
  "_id": ObjectId(),
  "idClient": ObjectId(),
  "idSuccursale": ObjectId(),
  "idTable": ObjectId(),
  "dateReservation": ISODate(),
  "heure": "18:00",
  "nombrePersonnes": 4,
  "statutReservation": "confirmée"
}
```

- **Plat**

```
{
  "_id": ObjectId(),
  "nomPlat": "Pizza",
  "description": "Pizza fromage",
  "prix": 15,
  "disponible": true,
  "idCategorie": ObjectId()
}
```

- **Ingredient**

```
{
  "_id": ObjectId(),
  "nomIngredient": "Fromage",
  "quantiteStock": 50,
  "seuilAlerte": 10,
  "uniteMesure": "kg",
  "idSuccursale": ObjectId()
}
```


- **CompositionPlat**

```
{
  "_id": ObjectId(),
  "idPlat": ObjectId(),
  "idIngredient": ObjectId(),
  "quantiteNecessaire": 0.2,
  "uniteMesure": "kg"
}
```

- **Livraison**

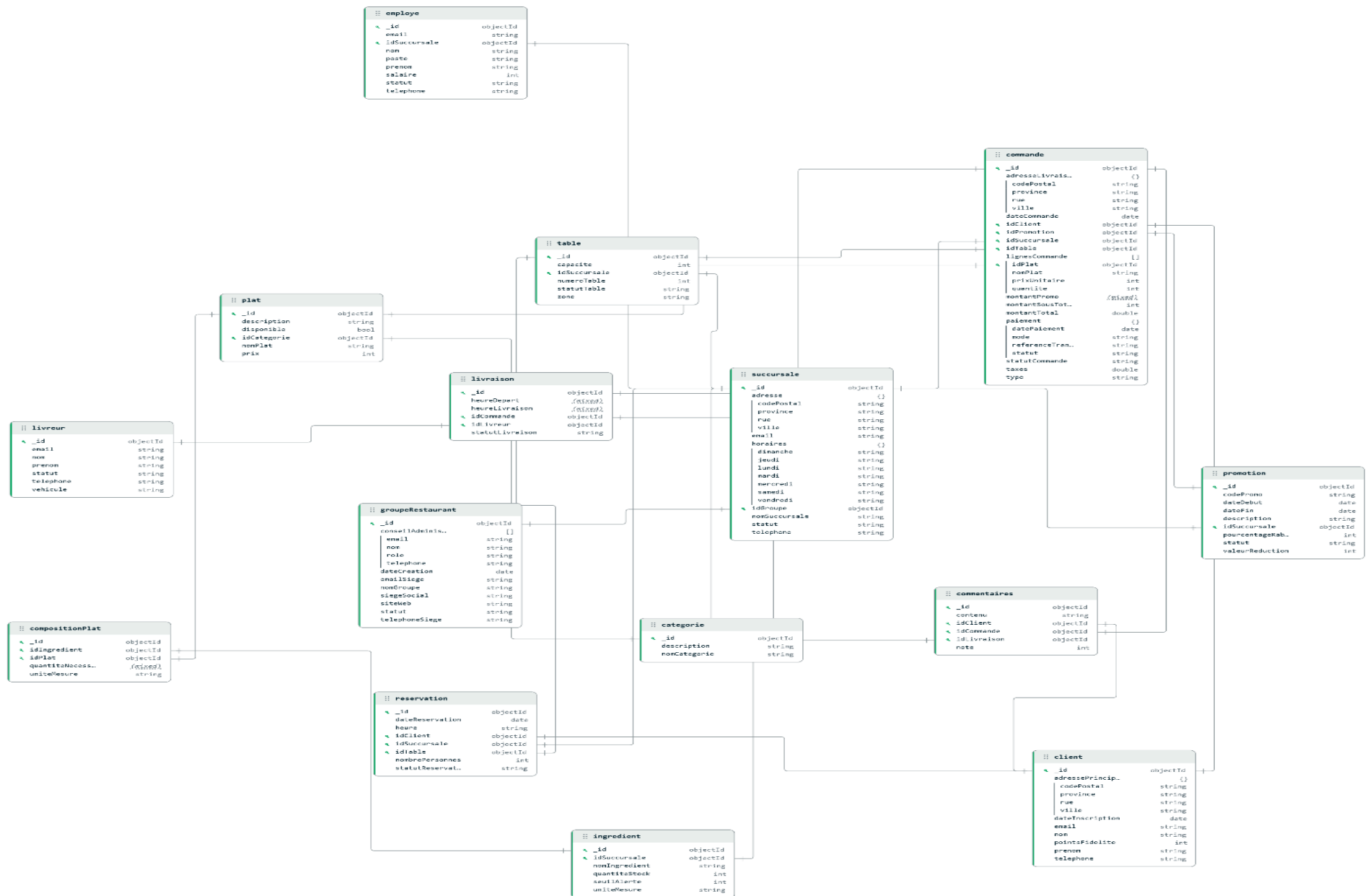
```
{
  "_id": ObjectId(),
  "idCommande": ObjectId(),
  "idLivreur": ObjectId(),
  "statutLivraison": "en_cours",
  "heureDepart": ISODate(),
  "heureLivraison": null
}
```

- **Commentaires**

```
{
  _id: ObjectId("com1"),
  idClient: ObjectId("client1"),
  idCommande: ObjectId("cmd1"),
  note: 4,
  contenu: "Service rapide et plat délicieux",
  dateCommentaire: ISODate("2026-03-20"),
  statut: "visible"
}
```

4. Implémentation

L'implémentation de la base de données a été réalisée à l'aide de scripts MongoDB Shell (mongosh). L'ensemble des commandes nécessaires à la création de la base de données, des collections, à l'insertion des données ainsi qu'à la configuration des différents éléments du projet est regroupé dans le fichier BD_LaTableCommune.txt, présenté en annexe.



Après la conception et l'implémentation de la base de données, la prochaine phase vise à exploiter les fonctionnalités avancées de MongoDB afin d'améliorer les performances, la sécurité et la capacité d'évolution du système. Il s'agira notamment de :

- optimiser les requêtes grâce à l'indexation ;
- exploiter les données à l'aide de pipelines d'agrégation ;
- mettre en œuvre une architecture distribuée basée sur la réplication et le sharding ;
- renforcer la sécurité par la validation du schéma et la gestion des utilisateurs et des rôles (RBAC).

Ces améliorations permettent de faire évoluer la base de données vers une solution performante, sécurisée et évolutive, adaptée à un environnement de production.

5. Optimisation des performances

5.1. Création d'index

Les index ont sont définis à partir de deux critères principaux : les règles métier imposant l'unicité de certaines données et les requêtes les plus fréquentes identifiées lors de l'analyse des charges. Celles-ci concernent notamment la consultation des commandes d'un client, le filtrage par succursale, le suivi des livraisons, la consultation des plats par catégorie et la gestion des réservations.

Par ailleurs, plusieurs champs de référence tels que `idClient`, `idSuccursale`, `idCommande`, `idCategorie` et `idTable` sont régulièrement utilisés dans les requêtes, ce qui en fait des candidats naturels à l'indexation.

Index unique sur `client.email`

Cet index garantit qu'un même client ne peut pas être enregistré deux fois avec le même courriel. Il répond à une contrainte d'intégrité métier et accélère aussi les recherches d'un client par email, par exemple lors d'une authentification, d'une vérification d'existence ou d'une consultation ciblée.

Index unique sur `promotion.codePromo`

Le code promotionnel doit être unique dans le système. Cet index permet donc d'assurer cette unicité tout en accélérant l'application d'une promotion lors d'une commande, puisque la recherche d'une promotion se fait naturellement à partir de son code.

Index simple sur commande.idClient

Un client pouvant effectuer plusieurs commandes, la consultation de son historique constitue une opération fréquente. L'index sur idClient permet d'accélérer cette recherche en évitant un parcours complet de la collection commande.

Index simple sur commande.idSuccursale

Cet index est utile pour les requêtes de filtrage et d'analyse par succursale, par exemple pour produire des statistiques de ventes ou retrouver les commandes associées à une succursale donnée. Comme la succursale est une référence centrale dans le modèle, ce champ est régulièrement sollicité.

Index composé sur commande.idClient et commande.dateCommande

Cet index composé est particulièrement pertinent pour le projet, car il permet non seulement de filtrer les commandes d'un client, mais aussi de les trier par date. Il est donc très utile pour des requêtes du type : « afficher les commandes récentes d'un client ». C'est un meilleur choix qu'un simple index sur idClient lorsque l'on veut à la fois filtrer et ordonner les résultats.

Index unique sur livraison.idCommande

Dans notre modèle, une commande en ligne donne lieu à une seule livraison. Un index unique sur idCommande dans la collection livraison permet donc de faire respecter cette relation 1:1 tout en accélérant la recherche d'une livraison à partir d'une commande.

Index composé sur reservation.idSuccursale, reservation.dateReservation, reservation.heureReservation et reservation.idTable

Le système doit gérer les réservations et vérifier la disponibilité des tables. Cet index composé est adapté aux recherches du type : « quelles réservations existent pour telle succursale, à telle date, à telle heure, sur telle table ? ». Il améliore donc les vérifications de disponibilité et la détection des conflits de réservation.

Index composé sur plat.idCategorie et plat.disponible

L'analyse des charges mentionne la consultation du menu par catégorie ainsi que la vérification de la disponibilité d'un plat. Cet index facilite l'affichage des plats disponibles dans une catégorie donnée, ce qui correspond à un cas d'usage très fréquent dans un système de restauration.

Les commandes MongoDB permettant de créer les index sont présentées ci-dessous.

```
use LaTableCommune

db.client.createIndex({ email: 1 }, { unique: true })

db.promotion.createIndex({ codePromo: 1 }, { unique: true })

db.commande.createIndex({ idClient: 1 })

db.commande.createIndex({ idSuccursale: 1 })

db.commande.createIndex({ idClient: 1, dateCommande: -1 })

db.livraison.createIndex({ idCommande: 1 }, { unique: true })

db.reservation.createIndex({
  idSuccursale: 1,
  dateReservation: 1,
  heureReservation: 1,
  idTable: 1
})

db.plat.createIndex({
  idCategorie: 1,
  disponible: 1
})
```

De manière générale :

- les **index uniques** garantissent l'unicité de certaines données conformément aux règles métier ;
- les **index simples** améliorent les performances des recherches et des filtres les plus fréquents ;
- les **index composés** optimisent les requêtes combinant plusieurs critères de filtrage et/ou de tri.

Le tableau suivant résume les principaux index mis en place ainsi que leur pertinence dans le cadre du projet.

Collection	Indexes	Type	Pertinence
client	{ email: 1 }	unique	éviter les doublons et accélérer la recherche d'un client par courriel
promotion	{ codePromo: 1 }	unique	garantir l'unicité des codes promotionnels
commande	{ idClient: 1 }	simple	retrouver rapidement les commandes d'un client
commande	{ idSuccursale: 1 }	simple	filtrage et statistiques par succursale
commande	{ idClient: 1, dateCommande: -1 }	composé	afficher les commandes d'un client triées par date
livraison	{ idCommande: 1 }	unique	respecter la relation 1:1 entre commande en ligne et livraison
reservation	{ idSuccursale: 1, dateReservation: 1, heureReservation: 1, idTable: 1 }	composé	vérifier la disponibilité d'une table pour une date et une heure données
plat	{ idCategorie: 1, disponible: 1 }	composé	afficher les plats disponibles par catégorie

Source : Élaboration de l'auteur

5.2. Démonstration de requête avant et après l'indexation avec la commande `explain()` pour démontrer l'impact sur les performances.

Nous allons baser la démonstration sur la requête suivante :

Requête : Filtre par `idClient` et de tri par `dateCommande` :
Retrouver les commandes d'un client, triées par date décroissant

```
db.commande.find({
  idClient: ObjectId("69de63067890f5c00ab28bfa")
})
.sort({ dateCommande: -1 })
.explain("executionStats")
```

Démonstration

Étape 1 : Exécution de la requête avant index avec la commande `explain()`

```
db.commande.find({
  idClient: ObjectId("69de63067890f5c00ab28bfa")
})
.sort({ dateCommande: -1 })
.explain("executionStats")
```

```
> db.commande.find({
  idClient: ObjectId("69de63067890f5c00ab28bfa")
})
.sort({ dateCommande: -1 })
.explain("executionStats")
< {
  explainVersion: '1',
  queryPlanner: {
    namespace: 'LaTableCommune.commande',
    parsedQuery: {
      idClient: {
        '$eq': ObjectId('69de63067890f5c00ab28bfa')
      }
    },
    indexFilterSet: false,
    queryHash: '13D70104',
    planCacheShapeHash: '13D70104',
    planCacheKey: 'E3CD612E',
    optimizationTimeMillis: 0,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    prunedSimilarIndexes: false,
    winningPlan: {
      isCached: false,
      stage: 'SORT',
      sortPattern: {
        dateCommande: -1
      },
    },
    memLimit: 104857600,
    type: 'simple',
```



```

        type: 'simple',
        inputStage: {
          stage: 'COLLSCAN',
          filter: {
            idClient: {
              '$eq': ObjectId('69de63067890f5c00ab28bfa')
            }
          },
          direction: 'forward'
        }
      },
      rejectedPlans: []
    },
    executionStats: {
      executionSuccess: true,
      nReturned: 6691,
      executionTimeMillis: 9,
      totalKeysExamined: 0,
      totalDocsExamined: 20020,
      executionStages: {
        isCached: false,
        stage: 'SORT',
        nReturned: 6691,
        executionTimeMillisEstimate: 0,
        works: 26713,
        advanced: 6691,
        needTime: 20021,
        needYield: 0,
        saveState: 0,
        restoreState: 0,
        isEOF: 1
      }
    }
  }
}

```

```

isEOF: 1,
sortPattern: {
  dateCommande: -1
},
memLimit: 104857600,
type: 'simple',
totalDataSizeSorted: 1271290,
usedDisk: false,
spills: 0,
spilledRecords: 0,
spilledBytes: 0,
spilledDataStorageSize: 0,
inputStage: {
  stage: 'COLLSCAN',
  filter: {
    idClient: {
      '$eq': ObjectId('69de63067890f5c00ab28bfa')
    }
  },
},
nReturned: 6691,
executionTimeMillisEstimate: 0,
works: 20021,
advanced: 6691,
needTime: 13329,
needYield: 0,
saveState: 0,
restoreState: 0,
isEOF: 1,
direction: 'forward',
docsExamined: 20020
}

```

```

    }
  },
  queryShapeHash: '8213EAB21CEDDB699E31B3B0ECD8F115C838D10E77116CD399570D2D39DF49EA',
  command: {
    find: 'commande',
    filter: {
      idClient: ObjectId('69de63067890f5c00ab28bfa')
    },
    sort: {
      dateCommande: -1
    },
    '$db': 'LaTableCommune'
  },
  serverInfo: {
    host: 'IKPOVENON',
    port: 27017,
    version: '8.2.3',
    gitVersion: '36f41c9c30a2f13f834d033ba03c3463c891fb01'
  },
  serverParameters: {
    internalQueryFacetBufferSizeBytes: 104857600,
    internalQueryFacetMaxOutputDocSizeBytes: 104857600,
    internalLookupStageIntermediateDocumentMaxSizeBytes: 104857600,
    internalDocumentSourceGroupMaxMemoryBytes: 104857600,
    internalQueryMaxBlockingSortMemoryUsageBytes: 104857600,
    internalQueryProhibitBlockingMergeOnMongoS: 0,
    internalQueryMaxAddToSetBytes: 104857600,
    internalDocumentSourceSetWindowFieldsMaxMemoryBytes: 104857600,
    internalQueryFrameworkControl: 'trySbeRestricted',
    internalQueryPlannerIgnoreIndexWithCollationForRegex: 1
  },
},
ok: 1
}

```

Zoom sur le Résultat avant index :

- ❖ Avant l'indexation, l'analyse de la requête avec `explain("executionStats")` montre que MongoDB n'utilise aucun index (`totalKeysExamined: 0`). La base doit examiner 20 020 documents (**`totalDocsExamined: 20020`**), ce qui correspond à un parcours quasi complet de la collection. De plus, une étape de **tri (SORT)** est nécessaire pour ordonner les résultats selon `dateCommande`. Cela démontre que la requête avant indexation est peu optimisée.

```

executionStats: {
  executionSuccess: true,
  nReturned: 6691,
  executionTimeMillis: 9,
  totalKeysExamined: 0,
  totalDocsExamined: 20020,
  executionStages: {
    isCached: false,
    stage: 'SORT',
  }
}

```

Etape 2 : Creation de l'index

```
db.commande.createIndex({ idClient: 1, dateCommande: -1 })
```

```

> db.commande.createIndex({ idClient: 1, dateCommande: -1 })
< idClient_1_dateCommande_-1

```

Etape 3 : Exécution de la requête après index avec la commande explain()

```

db.commande.find({
  idClient: ObjectId("69bd2d95c041bcfe7b628cb5")
})
.sort({ dateCommande: -1 })
.explain("executionStats")

```

```

> db.commande.createIndex({ idClient: 1, dateCommande: -1 })
< idClient_1_dateCommande_-1
> db.commande.find({
  idClient: ObjectId("69de63067890f5c00ab28bfa")
})
.sort({ dateCommande: -1 })
.explain("executionStats")

```

```

> db.commande.find({
  idClient: ObjectId("69de63067890f5c00ab28bfa")
})
.sort({ dateCommande: -1 })
.explain("executionStats")
< {
  explainVersion: '1',
  queryPlanner: {
    namespace: 'LaTableCommune.commande',
    parsedQuery: {
      idClient: {
        '$eq': ObjectId('69de63067890f5c00ab28bfa')
      }
    },
    indexFilterSet: false,
    queryHash: '13D70104',
    planCacheShapeHash: '13D70104',
    planCacheKey: 'BE6CE832',
    optimizationTimeMillis: 0,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    prunedSimilarIndexes: false,
    winningPlan: {
      isCached: false,
      stage: 'FETCH',
      inputStage: {
        stage: 'IXSCAN',
        keyPattern: {
          idClient: 1,
          dateCommande: -1

```

```

        dateCommande: -1
    },
    indexName: 'idClient_1_dateCommande_-1',
    isMultiKey: false,
    multiKeyPaths: {
        idClient: [],
        dateCommande: []
    },
    isUnique: false,
    isSparse: false,
    isPartial: false,
    indexVersion: 2,
    direction: 'forward',
    indexBounds: {
        idClient: [
            "[ObjectId('69de63067890f5c00ab28bfa'), ObjectId('69de63067890f5c00ab28bfa')]"
        ],
        dateCommande: [
            '[MaxKey, MinKey]'
        ]
    }
},
rejectedPlans: []
},
executionStats: {
    executionSuccess: true,
    nReturned: 6691,
    executionTimeMillis: 6,
    totalKeysExamined: 6691,
    totalDocsExamined: 6691,

```

```
executionStages: {
  isCached: false,
  stage: 'FETCH',
  nReturned: 6691,
  executionTimeMillisEstimate: 0,
  works: 6692,
  advanced: 6691,
  needTime: 0,
  needYield: 0,
  saveState: 0,
  restoreState: 0,
  isEOF: 1,
  docsExamined: 6691,
  alreadyHasObj: 0,
  inputStage: {
    stage: 'IXSCAN',
    nReturned: 6691,
    executionTimeMillisEstimate: 0,
    works: 6692,
    advanced: 6691,
    needTime: 0,
    needYield: 0,
    saveState: 0,
    restoreState: 0,
    isEOF: 1,
    keyPattern: {
      idClient: 1,
      dateCommande: -1
    },
    indexName: 'idClient_1_dateCommande_-1',
    isMultiKey: false,
```

```

    isMultiKey: false,
    multiKeyPaths: {
      idClient: [],
      dateCommande: []
    },
    isUnique: false,
    isSparse: false,
    isPartial: false,
    indexVersion: 2,
    direction: 'forward',
    indexBounds: {
      idClient: [
        "[ObjectId('69de63067890f5c00ab28bfa'), ObjectId('69de63067890f5c00ab28bfa')]"
      ],
      dateCommande: [
        '[MaxKey, MinKey]'
      ]
    },
    keysExamined: 6691,
    seeks: 1,
    dupsTested: 0,
    dupsDropped: 0
  }
},
queryShapeHash: '8213EAB21CEDDB699E31B3B0ECD8F115C838D10E77116CD399570D2D39DF49EA',
command: {
  find: 'commande',
  filter: {
    idClient: ObjectId('69de63067890f5c00ab28bfa')
  },

```



```

command: {
  find: 'commande',
  filter: {
    idClient: ObjectId('69de63067890f5c00ab28bfa')
  },
  sort: {
    dateCommande: -1
  },
  '$db': 'LaTableCommune'
},
serverInfo: {
  host: 'IKPOVENON',
  port: 27017,
  version: '8.2.3',
  gitVersion: '36f41c9c30a2f13f834d033ba03c3463c891fb01'
},
serverParameters: {
  internalQueryFacetBufferSizeBytes: 104857600,
  internalQueryFacetMaxOutputDocSizeBytes: 104857600,
  internalLookupStageIntermediateDocumentMaxSizeBytes: 104857600,
  internalDocumentSourceGroupMaxMemoryBytes: 104857600,
  internalQueryMaxBlockingSortMemoryUsageBytes: 104857600,
  internalQueryProhibitBlockingMergeOnMongoS: 0,
  internalQueryMaxAddToSetBytes: 104857600,
  internalDocumentSourceSetWindowFieldsMaxMemoryBytes: 104857600,
  internalQueryFrameworkControl: 'trySbeRestricted',
  internalQueryPlannerIgnoreIndexWithCollationForRegex: 1
},
ok: 1
}

```

LaTableCommune> |

Zoom sur le Résultat après index :

```
executionStats: {
  executionSuccess: true,
  nReturned: 6691,
  executionTimeMillis: 6,
  totalKeysExamined: 6691,
  totalDocsExamined: 6691,
  executionStages: {
    isCached: false,
    stage: 'FETCH',
    nReturned: 6691,
    executionTimeMillisEstimate: 0,
    works: 6692,
    advanced: 6691,
    needTime: 0,
    needYield: 0,
    saveState: 0,
    restoreState: 0,
    isEOF: 1,
    docsExamined: 6691,
    alreadyHasObj: 0,
    inputStage: {
      stage: 'IXSCAN',
```

1. nReturned: 6691

La requête retourne toujours **6691 documents**.

L'index ne change pas le résultat, il change seulement la manière de le trouver.

2. executionTimeMillis: 6

Le temps passe de **9 ms à 6 ms**.

Il y a une amélioration du temps d'exécution. Donc la réduction du travail effectué par MongoDB.

3. totalKeysExamined: 6691

MongoDB a examiné **6691 clés d'index**.

Cela prouve qu'il utilise bien un index.

4. totalDocsExamined: 6691

MongoDB a examiné **6691 documents**, soit uniquement les documents utiles.

Avant, il examinait **20020 documents**.

Après, il n'examine plus que **6691 documents**.

Donc MongoDB ne parcourt plus toute la collection.

5. stage: 'FETCH' avec inputStage: 'IXSCAN'

- IXSCAN = MongoDB utilise l'index pour localiser les documents
- FETCH = MongoDB va ensuite chercher les documents correspondants

Donc, après indexation :

- MongoDB ne fait plus un scan complet
- il s'appuie d'abord sur l'index
- puis récupère seulement les documents trouvés

Avant index

- totalKeysExamined: 0
- totalDocsExamined: 20020
- présence de SORT
- scan complet de la collection
- temps : **9 ms**

Après index

- totalKeysExamined: 6691
- totalDocsExamined: 6691
- présence de IXSCAN
- plus besoin de parcourir toute la collection
- temps : **6 ms**

6. Analyse des données

6.1. Pipelines d'Agrégation

Dans notre projet, nous n'avions pas encore implémenté de pipeline d'agrégation dans la section des requêtes. Nous avons donc ajouté une agrégation permettant d'obtenir un résumé des ventes par succursale. Ce choix est pertinent, car la collection commande contient les informations nécessaires pour analyser l'activité commerciale du restaurant, notamment idSuccursale, statutCommande et montantTotal. La collection succursale permet ensuite d'enrichir les résultats avec les informations descriptives de chaque succursale.

Objectif

Obtenir, pour chaque succursale :

- le nombre total de commandes

- le chiffre d'affaires total
- le montant moyen des commandes
- le tout trié par chiffre d'affaires décroissant

Pipeline avec étape avancée

```
db.commande.aggregate([
  {
    $match: {
      statutCommande: "terminee"
    }
  },
  {
    $group: {
      _id: "$idSuccursale",
      nombreCommandes: { $sum: 1 },
      chiffreAffairesTotal: { $sum: "$montantTotal" },
      moyenneMontantCommande: { $avg: "$montantTotal" }
    }
  },
  {
    $lookup: {
      from: "succursale",
      localField: "_id",
      foreignField: "_id",
      as: "infosSuccursale"
    }
  },
  {
    $unwind: "$infosSuccursale"
  },
  {
    $project: {
      _id: 0,
```

```

        idSuccursale: "$_id",
        nomSuccursale: "$infosSuccursale.nomSuccursale",
        ville: "$infosSuccursale.ville",
        nombreCommandes: 1,
        chiffreAffairesTotal: 1,
        moyenneMontantCommande: { $round:
["$moyenneMontantCommande", 2] }
    }
},
{
    $sort: {
        chiffreAffairesTotal: -1
    }
}
])

```

Explication :

Explication Des étapes

1. \$match

```

{
  $match: {
    statutCommande: "terminee"
  }
}

```

Cette étape filtre les commandes pour ne garder que celles qui sont terminées. Ceci permet d'analyser de vraies ventes finalisées, pas les commandes annulées ou encore en cours.

Rôle classique de \$match : Filtrer les documents comme un WHERE en SQL.

2. \$group

```

{
  $group: {
    _id: "$idSuccursale",
    nombreCommandes: { $sum: 1 },
    chiffreAffairesTotal: { $sum: "$montantTotal" },
    moyenneMontantCommande: { $avg: "$montantTotal" }
  }
}

```

```
}  
}
```

Cette étape regroupe les commandes par succursale. Elle permet de calculer :

- le nombre de commandes
- le chiffre d'affaires total
- la moyenne du montant des commandes

Rôle classique de \$group : Regrouper les documents et effectuer des calculs agrégés :

- somme (\$sum)
- moyenne (\$avg)
- minimum / maximum
- comptage

Equivalent de GROUP BY en SQL

3. \$lookup étape avancée

```
{  
  $lookup: {  
    from: "succursale",  
    localField: "_id",  
    foreignField: "_id",  
    as: "infosSuccursale"  
  }  
}
```

Cette étape avancée fait une jointure entre les résultats d'agrégation et la collection succursale.

Dans notre contexte, elle sert à récupérer les informations de la succursale correspondant à chaque groupe, par exemple son nom.

Role classique : L'opérateur \$lookup permet d'effectuer une jointure entre deux collections MongoDB. Il enrichit les documents en ajoutant des données provenant d'une autre collection, similaire à une opération JOIN en SQL.

4. \$unwind étape avancée

```
{
  $unwind: "$infosSuccursale"
}
```

En fait, après \$lookup, les informations de la succursale sont placées dans un tableau.

\$unwind sert à transformer ce tableau en objet simple pour faciliter l’affichage et la projection. Ici, cela permet d’obtenir directement nomSuccursale au lieu d’un tableau infosSuccursale.

Rôle classique : L’opérateur \$unwind permet de décomposer un tableau en plusieurs documents, chacun contenant un seul élément du tableau. Cela facilite le traitement et l’analyse des données imbriquées.

5. \$project

```
{
  $project: {
    _id: 0,
    idSuccursale: "$_id",
    nomSuccursale: "$infosSuccursale.nomSuccursale",
    ville: "$infosSuccursale.ville",
    nombreCommandes: 1,
    chiffreAffairesTotal: 1,
    moyenneMontantCommande: { $round:
["$moyenneMontantCommande", 2] }
  }
}
```

Cette étape permet de choisir les champs finaux à afficher et de rendre le résultat plus clair. Elle reformate aussi la sortie pour qu’elle soit plus professionnelle.

Rôle classique : L’opérateur \$project permet de sélectionner, renommer ou transformer les champs des documents en sortie du pipeline. Il est similaire à la clause SELECT en SQL.

6. \$sort

```
{  
  $sort: {  
    chiffreAffairesTotal: -1  
  }  
}
```

Cette étape trie les succursales de la plus performante à la moins performante selon le chiffre d'affaires total.

C'est utile pour une lecture rapide des résultats.

Rôle classique : L'opérateur \$sort permet de trier les documents selon un ou plusieurs champs, en ordre croissant ou décroissant. Il est équivalent à la clause ORDER BY en SQL.

7. \$sum

\$sum Calculer une somme ou compter

8. \$avg

\$avg Calculer une moyenne

9. \$round

\$round arrondir

Résultat du pipeline :

```
< {  
  nombreCommandes: 6737,  
  chiffreAffairesTotal: 100842,  
  idSuccursale: ObjectId('69de635d7890f5c00ab28bfd'),  
  nomSuccursale: 'Ottawa',  
  moyenneMontantCommande: 14.97  
}  
{  
  nombreCommandes: 6647,  
  chiffreAffairesTotal: 99732,  
  idSuccursale: ObjectId('69de635d7890f5c00ab28bfb'),  
  nomSuccursale: 'Aylmer',  
  moyenneMontantCommande: 15  
}  
{  
  nombreCommandes: 6636,  
  chiffreAffairesTotal: 99621,  
  idSuccursale: ObjectId('69de635d7890f5c00ab28bfc'),  
  nomSuccursale: 'Hull',  
  moyenneMontantCommande: 15.01  
}  
LaTableCommune >
```

6.2. Réplication & Sharding

Cette section présente les concepts de réplication et de sharding dans MongoDB à l'aide d'exemples de configuration en environnement local. Bien que les démonstrations soient réalisées localement pour des raisons pédagogiques, **le projet a été effectivement déployé et validé sur MongoDB Atlas, où la réplication, le sharding et la haute disponibilité sont pris en charge nativement par la plateforme.**

5. Configuration d'un Replica Set en local

La réplication consiste à créer des copies redondantes des données sur plusieurs serveurs pour garantir la disponibilité des données même en cas de panne d'un serveur. Un Replica Set est un groupe de serveurs MongoDB contenant :

- Un primaire (Primary) : Accepte les écritures et réplique les données vers les secondaires
- Un ou plusieurs secondaires (Secondaries) : Reçoivent des copies des données du primaire et peuvent servir les lectures

Étape 1 : Créer les répertoires de données

Il est nécessaire de créer des répertoires distincts pour chaque membre du Replica Set :

```
mkdir C:\data\Primaire
```

```
mkdir C:\data\Secondaire1
```

```
mkdir C:\data\Secondaire2
```

Étape 2 : Lancer les instances mongod

Selon les paramètres du projet, chaque instance est lancée avec le nom de Replica Set testrepli.

```
# Instance Primaire (port 2717)
```

```
mongod --port 2717 --dbpath "C:\data\Primaire" --replSet "testrepli"
```

```
# Instance Secondaire 1 (port 2727)
```

```
mongod --port 2727 --dbpath "C:\data\Secondaire1" --replSet "testrepli"
```

```
# Instance Secondaire 2 (port 2737)
```

```
mongod --port 2737 --dbpath "C:\data\Secondaire2" --replSet "testrepli"
```

Étape 3 : Initialisation et vérification

Connexion à l'instance principale pour initialiser le groupe.

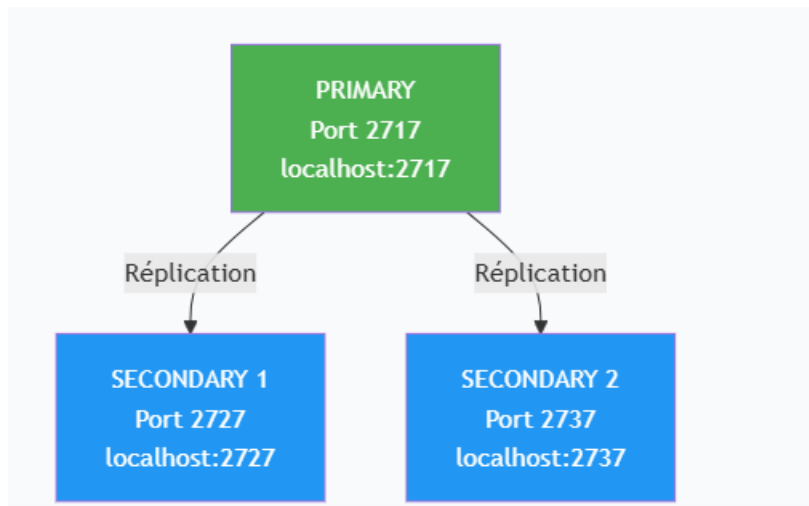
```
mongosh --host "localhost:2717"
```

```
// Initialisation avec les membres spécifiés
```

```
rs.initiate({  
  _id: "testrepli",  
  members: [  
    { _id: 0, host: "localhost:2717" },  
    { _id: 1, host: "localhost:2727" },  
    { _id: 2, host: "localhost:2737" }  
  ]  
})
```

// Vérification de l'état

rs.status()



État du Replica Set (rs.status())

Les écrans ci-dessous montrent le statut du replica set

```
members: [
  {
    _id: 0,
    name: 'localhost:2717',
    health: 1,
    state: 1,
    stateStr: 'PRIMARY',
    uptime: 292,
    optime: { ts: Timestamp({ t: 1776224348, i: 1 }), t: Long('1') },
    optimeDate: ISODate('2026-04-15T03:39:08.000Z'),
    optimeWritten: { ts: Timestamp({ t: 1776224348, i: 1 }), t: Long('1') },
    optimeWrittenDate: ISODate('2026-04-15T03:39:08.000Z'),
    lastAppliedWallTime: ISODate('2026-04-15T03:39:08.972Z'),
    lastDurableWallTime: ISODate('2026-04-15T03:39:08.972Z'),
    lastWrittenWallTime: ISODate('2026-04-15T03:39:08.972Z'),
    syncSourceHost: '',
    syncSourceId: -1,
    infoMessage: 'Could not find member to sync from',
    electionTime: Timestamp({ t: 1776224268, i: 1 }),
    electionDate: ISODate('2026-04-15T03:37:48.000Z'),
    configVersion: 1,
    configTerm: 1,
    self: true,
    lastHeartbeatMessage: ''
  },
]
```

```
{
  _id: 1,
  name: 'localhost:2727',
  health: 1,
  state: 2,
  stateStr: 'SECONDARY',
  uptime: 92,
  optime: { ts: Timestamp({ t: 1776224348, i: 1 }), t: Long('1') },
  optimeDurable: { ts: Timestamp({ t: 1776224348, i: 1 }), t: Long('1') },
  optimeWritten: { ts: Timestamp({ t: 1776224348, i: 1 }), t: Long('1') },
  optimeDate: ISODate('2026-04-15T03:39:08.000Z'),
  optimeDurableDate: ISODate('2026-04-15T03:39:08.000Z'),
  optimeWrittenDate: ISODate('2026-04-15T03:39:08.000Z'),
  lastAppliedWallTime: ISODate('2026-04-15T03:39:08.972Z'),
  lastDurableWallTime: ISODate('2026-04-15T03:39:08.972Z'),
  lastWrittenWallTime: ISODate('2026-04-15T03:39:08.972Z'),
  lastHeartbeat: ISODate('2026-04-15T03:39:09.239Z'),
  lastHeartbeatRecv: ISODate('2026-04-15T03:39:08.213Z'),
  pingMs: Long('0'),
  lastHeartbeatMessage: '',
  syncSourceHost: 'localhost:2717',
  syncSourceId: 0,
  infoMessage: '',
  configVersion: 1,
  configTerm: 1
},
```

```
{
  _id: 2,
  name: 'localhost:2737',
  health: 1,
  state: 2,
  stateStr: 'SECONDARY',
  uptime: 92,
  optime: { ts: Timestamp({ t: 1776224348, i: 1 }), t: Long('1') },
  optimeDurable: { ts: Timestamp({ t: 1776224348, i: 1 }), t: Long('1') },
  optimeWritten: { ts: Timestamp({ t: 1776224348, i: 1 }), t: Long('1') },
  optimeDate: ISODate('2026-04-15T03:39:08.000Z'),
  optimeDurableDate: ISODate('2026-04-15T03:39:08.000Z'),
  optimeWrittenDate: ISODate('2026-04-15T03:39:08.000Z'),
  lastAppliedWallTime: ISODate('2026-04-15T03:39:08.972Z'),
  lastDurableWallTime: ISODate('2026-04-15T03:39:08.972Z'),
  lastWrittenWallTime: ISODate('2026-04-15T03:39:08.972Z'),
  lastHeartbeat: ISODate('2026-04-15T03:39:09.239Z'),
  lastHeartbeatRecv: ISODate('2026-04-15T03:39:08.196Z'),
  pingMs: Long('0'),
  lastHeartbeatMessage: '',
  syncSourceHost: 'localhost:2717',
  syncSourceId: 0,
  infoMessage: '',
  configVersion: 1,
  configTerm: 1
}
```

La capture d'écran ci-dessus montre le résultat de la commande `rs.status()` exécutée après l'initialisation du Replica Set. On peut y observer les informations suivantes :

- "set" : "testrepli" : Le Replica Set a bien été créé avec le nom que nous avons défini.

- Membre localhost:2717 : Son stateStr est "PRIMARY" et son health est à 1, ce qui signifie qu'il est en bonne santé et qu'il accepte les écritures.
- Membres localhost:2727 et localhost:2737 : Leur stateStr est "SECONDARY" et leur health est également à 1, ce qui confirme qu'ils répliquent correctement les données du PRIMARY.
- syncSourceHost: 'localhost:2717' : Les deux SECONDARY indiquent qu'ils se synchronisent bien à partir du PRIMARY (port 2717).

Cette configuration garantit la haute disponibilité de notre base de données La Table Commune : si le PRIMARY venait à tomber en panne, un SECONDARY serait automatiquement élu pour prendre le relais.

6. Configuration du Sharding

Le sharding permet de diviser les données en fragments répartis sur plusieurs clusters pour gérer de gros volumes.

Étape 1 : Configuration des instances des shards

```
mongod --shardsvr --port 2710 --dbpath "C:\data\Shard1" --replSet "testshard"
```

```
mongod --shardsvr --port 2720 --dbpath "C:\data\Shard2" --replSet "testshard"
```

```
mongod --shardsvr --port 2730 --dbpath "C:\data\Shard3" --replSet "testshard"
```

Étape 2 : Lancement du Routeur (mongos)

Le processus mongos aiguille les requêtes vers les bons shards via les config servers.

```
mongos --port 1210 --configdb testrepli/localhost:2717,localhost:2727,localhost:2737
```

Étape 3 : Ajout des Shards au cluster

```
mongosh --host "localhost:1210"
```

```
sh.addShard("testshard/localhost:2710,localhost:2720,localhost:2730")
```

Ecran 2 : Configuration du Sharding

La capture d'écrans ci-dessus illustre deux étapes clés de la configuration du sharding pour le projet

Lancement du routeur mongos

La première partie de la capture montre le lancement du routeur mongos avec la commande :

```
mongos --port 1210 --configdb
```

configRepl/localhost:2717,localhost:2727,localhost:2737

Le routeur se connecte aux Config Servers (Replica Set configRepl) pour obtenir les métadonnées du cluster. Le message "Waiting for connections" confirme que le routeur est opérationnel sur le port 1210.

```
Microsoft Windows [version 10.0.20200.8037]
(c) Microsoft Corporation. All rights reserved.

C:\Users\jeanr>mongosh --port 1210
Current Mongosh Log ID: 69df1386a1103f79cd628c9f
Connecting to:      mongodb://127.0.0.1:1210/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.6.0
Using MongoDB:      8.2.3
Using Mongosh:       2.6.0
mongosh 2.8.2 is available for download: https://www.mongodb.com/try/download/shell

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

-----
  The server generated these startup warnings when booting
  2026-04-15T00:24:45.750-04:00: Access control is not enabled for the database. Read and write access to data and configuration is
unrestricted
  2026-04-15T00:24:45.751-04:00: This server is bound to localhost. Remote systems will be unable to connect to this server. Start t
he server with --bind_ip <address> to specify which IP addresses it should serve responses from, or with --bind_ip_all to bind to all
interfaces. If this behavior is desired, start the server with --bind_ip 127.0.0.1 to disable this warning
-----

[direct: mongos] test> sh.addShard("shard1Repl/localhost:2710")
{
  shardAdded: 'shard1Repl',
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1776227287, i: 57 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1776227287, i: 57 })
}
```

La deuxième partie montre l'ajout des trois shards au cluster via les commandes :

```
sh.addShard("shard1Repl/localhost:2710")
```

```
sh.addShard("shard2Repl/localhost:2720")
```

```
sh.addShard("shard3Repl/localhost:2730")
```

Analyse des résultats :

Shard	Port	Résultat	Signification
shard1Repl	2710	{ ok: 1 }	Ajout réussi
shard2Repl	2720	{ ok: 1 }	Ajout réussi
shard3Repl	2730	{ ok: 1 }	Ajout réussi

Chaque retour { ok: 1 } confirme que le shard a été correctement ajouté au cluster. Les champs shardAdded indiquent le nom du Replica Set associé à chaque shard.

Cette configuration permet de distribuer les données sur plusieurs serveurs, garantissant ainsi l'extensibilité horizontale et la répartition de la charge.

```
mongosh mongodb://127.0.0.1:27020
> use test
test> sh.addShard("shard2Repl/localhost:2720")
{
  shardAdded: 'shard2Repl',
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1776227316, i: 52 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1776227316, i: 52 })
}
[direct: mongos] test> sh.addShard("shard3Repl/localhost:2730")
{
  shardAdded: 'shard3Repl',
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1776227347, i: 47 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1776227347, i: 47 })
}
[direct: mongos] test> |
```

7. Choix de la clé de sharding

Pour la collection principale commande dans la base LaTableCommune, nous activons le partitionnement.

```
sh.enableSharding("LaTableCommune")
sh.shardCollection("LaTableCommune.commande", { idSuccursale: 1, dateCommande: 1 })
```

Ecran 3 : Activation du sharding sur la collection

L'image de l'écran 3 montrant l'activation du sharding sur la collection


```

[direct: mongos] test> sh.enableSharding("LaTableCommune")
{
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1776228173, i: 21 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1776228173, i: 21 })
}
[direct: mongos] test> sh.shardCollection("LaTableCommune.commande", { idSuccursale: 1, dateCommande: 1 })
{
  collectionsSharded: 'LaTableCommune.commande',
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1776228199, i: 23 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1776228199, i: 22 })
}
[direct: mongos] test> |

```

La capture d'écran ci-dessus montre l'activation du partitionnement pour la base LaTableCommune et la collection commande :

`sh.enableSharding("LaTableCommune")`

`sh.shardCollection("LaTableCommune.commande", { idSuccursale: 1, dateCommande : 1 })`

Clé de sharding choisie : { idSuccursale: 1, dateCommande: 1 } (clé composite)

Justification du choix de la clé composite { idSuccursale, dateCommande } :

Critère	Explication
Uniformité de distribution	Les commandes se répartissent entre les différentes succursales, évitant la saturation d'un seul shard.
Requêtes fréquentes	Les analyses métier filtrent souvent par succursale ou par date, ce qui permet au routeur mongos de cibler directement les shards concernés.

Critère	Explication
Cardinalité	La combinaison des deux champs offre un grand nombre de valeurs uniques pour un découpage efficace en chunks.

Le retour { ok: 1 } pour chaque commande confirme que :

- La base LaTableCommune est désormais activée pour le sharding
- La collection commande est correctement fragmentée selon la clé composite choisie

Cette configuration permet à La Table Commune de distribuer les commandes sur plusieurs shards, garantissant ainsi l'extensibilité horizontale pour accompagner la croissance du restaurant.

Partie 4 – Schéma & Sécurité

4.1 Modification du schéma sans altérer les anciens documents

Dans MongoDB, le fait qu'il n'y ait pas de schéma fixe est l'un de ses grands avantages. On peut ajouter un nouveau champ à certains documents sans que cela ne pose le moindre problème aux documents déjà existants. Ils restent valides et accessibles normalement, même s'ils ne contiennent pas ce nouveau champ.

Dans le cadre du projet La Table Commune, nous avons décidé d'ajouter un champ `programme_fidelite` à la collection `client`. Ce champ permet de savoir si un client est inscrit au programme de fidélité du restaurant et quel est son niveau (standard, silver, gold). Ce besoin est apparu après la mise en production initiale de la base, ce qui rend ce cas particulièrement réaliste.

Étape 1 : Ajouter le nouveau champ `programme_fidelite` aux clients existants

On utilise la commande `updateMany()` combinée à l'opérateur `$set` pour ajouter ce champ à tous les documents existants de la collection `client`. Cela ne supprime aucun champ existant, ne casse aucun document, et les documents qui ne sont pas mis à jour restent simplement sans ce champ :

```
// Étape 1 : Ajouter le champ programme_fidelite à tous les clients existants
// On utilise $set pour créer ce champ sans toucher aux autres champs
db.client.updateMany(
  {},
  {
    $set: {
      programme_fidelite: {
        inscrit: false,
        niveau: "standard",
        points: 0
      }
    }
  }
);
```

```
// Résultat attendu :
// {
//   acknowledged: true,
//   matchedCount: 1500,
//   modifiedCount: 1500
// }
```

Le résultat retourné par MongoDB confirme que les 1500 documents de la collection client ont bien été mis à jour. Le champ modifiedCount égal à matchedCount indique qu'aucun document n'avait déjà ce champ, ce qui est normal puisque c'est un ajout

Étape 2 : Vérifier que les anciens documents restent valides

Pour s'assurer que l'ajout du champ n'a pas causé de problème, on peut interroger un document mis à jour et un document potentiellement non mis à jour avec find(). Les deux types de documents coexistent sans conflit dans la même collection.

```
LaTableCommune;> // Vérification : afficher un client après mise à jour
db.client.findOne({ email: 'marie.dupont@email.com' });
```

```
// Résultat attendu :
// {
//   _id: ObjectId('...'),
//   nom: 'Dupont',
//   prenom: 'Marie',
//   email: 'marie.dupont@email.com',
//   telephone: '613-555-0101',
//   adresses: [...],
//   programme_fidelite: { => nouveau champ ajouté
```

```
//    inscrit: false,
//    niveau: 'standard',
//    points: 0
//  }
// }
```

```
// Vérification : compter les documents SANS le nouveau champ
db.client.countDocuments({ programme_fidelite: { $exists: false } });|
```

// Résultat : 0 (tous les clients ont bien été mis à jour)

Étape 3 : Mise à jour ciblée pour un client qui rejoint le programme

Quand un client s'inscrit au programme de fidélité, on peut mettre à jour uniquement son document avec \$set et \$inc, sans toucher aux autres clients ni à la structure générale de la collection.

```
LaTableCommune;> // Un client s'inscrit au programme et accumule ses premiers points
db.client.updateOne(
  { email: 'marie.dupont@email.com' },
  {
    $set: {
      'programme_fidelite.inscrit': true,
      'programme_fidelite.niveau': 'silver',
      'programme_fidelite.date_inscription': new Date()
    },
    $inc: {
      'programme_fidelite.points': 250
    }
  }
);
```

```
LaTableCommune;> // Vérifier les modifications du client
db.client.findOne({ email: 'marie.dupont@email.com' });|
```

```
// Résultat attendu :
// {
//   acknowledged: true,
//   matchedCount: 1,
//   modifiedCount: 1
```

```
// }
```

Ce type de modification est totalement transparent pour les autres documents de la collection. MongoDB n'impose aucune contrainte de structure par défaut, donc un client avec `programme_fidelite.inscrit: true` et un client sans ce champ peuvent cohabiter sans aucun problème dans la même collection.

4.1.4 Ajout d'un validateur JSON Schema complet sur la collection client

Jusqu'ici, les modifications de schéma ont été réalisées sans contrainte formelle. Pour aller plus loin et assurer la cohérence des données, MongoDB permet d'associer un validateur JSON Schema à une collection. Ce validateur est une contrainte appliquée à chaque document inséré ou modifié : seuls les documents conformes aux règles définies seront acceptés. Les documents déjà présents dans la collection ne sont pas affectés si on choisit le niveau de validation `moderate`, ce qui permet d'introduire la validation progressivement sans casser les données existantes.

Dans le cadre du projet La Table Commune, nous appliquons un validateur complet sur la collection `client`. Ce validateur couvre les champs essentiels du document : les types de données (`bsonType`), les champs obligatoires (`required`), les contraintes de longueur (`minLength`, `maxLength`), les expressions régulières (`pattern`), les valeurs autorisées (`enum`), et les plages numériques (`minimum`, `maximum`). Voici les règles métier que ce validateur traduit en code :

- nom et prenom : chaînes de 2 à 50 caractères, obligatoires
- email : format valide `xxx@xxx.xxx`, obligatoire et unique
- telephone : format nord-américain (ex. 613-555-0101), optionnel
- dateNaissance : date, l'âge doit être représentable
- statut : valeur parmi `'actif'`, `'inactif'`, `'suspendu'` (`enum`)
- programme_fidelite.niveau : valeur parmi `'standard'`, `'silver'`, `'gold'` (`enum`)
- programme_fidelite.points : entier `>= 0`

Commande : Appliquer le validateur sur la collection existante

On utilise la commande `collMod` pour ajouter le validateur à une collection déjà existante. L'option `validationLevel: 'moderate'` garantit que les documents existants ne sont pas revalidés et que seuls les nouveaux documents ou les mises à jour doivent respecter le schéma.

```

LaTableCommune; > db.runCommand({
  collMod: "client",
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["nom", "prenom", "email", "statut"],
      properties: {
        // --- Identité ---
        nom: {
          bsonType: "string",
          minLength: 2,
          maxLength: 50,
          description: "Nom de famille obligatoire, entre 2 et 50 caractères"
        },
        prenom: {
          bsonType: "string",
          minLength: 2,
          maxLength: 50,
          description: "Prénom obligatoire, entre 2 et 50 caractères"
        },
        // --- Contact ---
        email: {
          bsonType: "string",
          pattern: "^[a-zA-Z0-9._%+\\-]+@[a-zA-Z0-9\\.\\-]+\\.([a-zA-Z]{2,})$",
          description: "Adresse courriel valide et obligatoire"
        },
        telephone: {
          bsonType: "string",
          pattern: "^[0-9]{3}-[0-9]{3}-[0-9]{4}$",
          description: "Format nord-américain : 514-555-0101"
        },
        // --- Statut du compte ---
        statut: {
          bsonType: "string",
          enum: ["actif", "inactif", "suspendu"],
          description: "Statut autorisé : actif, inactif ou suspendu"
        },
        // --- Date de naissance ---
        dateNaissance: {
          bsonType: "date",
          description: "Date de naissance sous forme de date MongoDB"
        },

```

```
// --- Programme de fidélité (objet imbriqué) ---
programme_fidelite: {
  bsonType: "object",
  properties: {
    inscrit: {
      bsonType: "bool"
    },
    niveau: {
      bsonType: "string",
      enum: ["standard", "silver", "gold"],
      description: "Niveau du programme : standard, silver ou gold"
    },
    points: {
      bsonType: "int",
      minimum: 0,
      description: "Les points ne peuvent pas être négatifs"
    },
    date_inscription: {
      bsonType: "date"
    }
  }
}
}
```

```
    } // fin properties
  } // fin $jsonSchema
}, // fin validator
validationLevel: "moderate",
validationAction: "error"
});
```

// Résultat attendu :

// { ok: 1 }

Le paramètre `validationLevel: 'moderate'` signifie que MongoDB ne revalidera pas les documents déjà existants. Seuls les nouveaux inserts et les updates sur des champs couverts par le schéma seront vérifiés. Le paramètre `validationAction: 'error'` fait en sorte que tout document non conforme est rejeté immédiatement avec un message d'erreur.

Démonstration : insertion valide (acceptée)

Le document ci-dessous respecte toutes les règles du validateur : les champs obligatoires sont présents, les formats sont corrects, le statut et le niveau de fidélité font partie des valeurs autorisées par enum.

```
LaTableCommune;> / Insert valide : tous les champs respectent les contraintes
db.client.insertOne({
  nom: "Tremblay",
  prenom: "Sophie",
  email: "sophie.tremblay@email.com",
  telephone: "613-555-0202",
  dateNaissance: ISODate("1992-06-15"),
  statut: "actif",
  programme_fidelite: {
    inscrit: true,
    niveau: "silver",
    points: NumberInt(320),
    date_inscription: ISODate("2024-01-10")
  }
});
```

```
// Résultat :
```

```
// { acknowledged: true, insertedId: ObjectId('...') }
```

Démonstration : insertions invalides (rejetées)

Les deux exemples ci-dessous montrent des documents qui violent les règles du validateur. MongoDB les rejette et retourne un message d'erreur qui identifie précisément quelle propriété n'est pas conforme.

```
LaTableCommune;> // Cas 1 : email manquant (champ required)
db.client.insertOne({
  nom: "Martin",
  prenom: "Luc",
  statut: "actif"
  // email absent → violation de required
});

// Cas 2 : statut non autorisé (violation enum)
db.client.insertOne({
  nom: "Beaulieu",
  prenom: "Anne",
  email: "anne.beaulieu@email.com",
  statut: "vip"
  // 'vip' n'est pas dans ['actif', 'inactif', 'suspendu']
});
```

```
// Document failed validation: required property 'email' is missing
```

```
// Document failed validation: 'statut' value is not one of the allowed values
```



```

// Cas 3 : points négatifs (violation minimum)
db.client.updateOne(
  { email: "sophie.tremblay@email.com" },
  { $set: { 'programme_fidelite.points': NumberInt(-50) } }
);

// Cas 4 : format téléphone invalide (violation pattern)
db.client.insertOne({
  nom: "Roy",
  prenom: "Paul",
  email: "paul.roy@email.com",
  telephone: "5145550101",
  // format attendu : 514-555-0101 avec tirets
  statut: "actif"
});

```

```

// Document failed validation: 'programme_fidelite.points' value -50 is less
than minimum 0

```

```

// Document failed validation: 'telephone' did not match the specified pattern

```

Ces quatre cas de rejet confirment que le validateur fonctionne correctement. Chaque contrainte définie dans le \$jsonSchema est bien appliquée : required bloque les champs manquants, enum empêche les valeurs non prévues, minimum rejette les valeurs numériques trop basses, et pattern filtre les formats de chaîne incorrects. Dans un contexte de restaurant, cela garantit l'intégrité des données clients dès la saisie, avant même que l'application n'intervienne.

4.2 Création d'un utilisateur MongoDB avec des droits limités

Dans un environnement de production, il est essentiel de ne pas utiliser un seul compte administrateur pour toutes les opérations. MongoDB propose un système de gestion des utilisateurs basé sur des rôles (RBAC : Role-Based Access Control). Chaque utilisateur se voit attribuer uniquement les permissions dont il a besoin, ce qu'on appelle le principe du moindre privilège.

Dans le projet La Table Commune, on va créer deux utilisateurs : un utilisateur applicatif (appUser) qui peut lire et écrire les données du restaurant, et un utilisateur en lecture seule (rapportUser) pour les rapports analytiques.

Étape 1 : Activer l'authentification dans mongod.cfg

Avant de créer des utilisateurs, il faut activer l'authentification dans le fichier de configuration de MongoDB. Sans cette étape, n'importe qui peut se connecter sans mot de passe.

```
# Dans le fichier mongod.cfg (Windows)
# Ajouter ou décommenter la section suivante :
security:
  authorization: enabled
# Redémarrer le service MongoDB après cette modification
```

Étape 2 : Créer l'utilisateur applicatif (lecture/écriture)

Cet utilisateur est celui utilisé par l'application du restaurant pour effectuer les opérations courantes : enregistrer une commande, mettre à jour une réservation, etc. On lui accorde le rôle readWrite uniquement sur la base LaTableCommune.

```
> use admin;
// Création de l'utilisateur applicatif
db.createUser({
  user: "appUser",
  pwd: "LaTable@2025!Secure",
  roles: [
    { role: "readWrite", db: "LaTableCommune" }
  ]
});
```

```
// Résultat :
// { ok: 1 }
```

Étape 3 : Créer l'utilisateur rapport (lecture seule)

Cet utilisateur est destiné aux membres de l'équipe de gestion qui consultent les rapports de ventes et les statistiques. Il peut lire les données mais ne peut pas les modifier. On lui attribue le rôle read uniquement.

```

use admin;
// Création de l'utilisateur pour les rapports (lecture seule)
db.createUser({
  user: "rapportUser",
  pwd: "Rapport@2025!ReadOnly",
  roles: [
    { role: "read", db: "LaTableCommune" }
  ]
});

```

```

// Résultat attendu :
// { ok: 1 }

```

Étape 4 : Démonstration d'une tentative d'action interdite

Pour démontrer que les restrictions fonctionnent correctement, on tente de faire une insertion avec l'utilisateur rapportUser, qui n'a que des droits de lecture. MongoDB doit refuser l'opération et retourner une erreur d'autorisation.

```

// Connexion avec l'utilisateur lecture seule
mongosh -u rapportUser -p 'Rapport@2025!ReadOnly' --
authenticationDatabase admin

```

```

> // Tentative d'insertion (action interdite pour rapportUser)
db.client.insertOne({
  nom: "Joseph",
  prenom: "Jean Robert",
  email: "jrjoseph@mail.com"
});

```

```

// MongoServerError: not authorized on LaTableCommune
// to execute command { insert: 'client', ... }

// En revanche, une lecture fonctionne normalement :
db.client.findOne(); // Retourne un document client sans erreur

```

Cette démonstration confirme que le principe du moindre privilège est bien appliqué. L'utilisateur rapportUser peut consulter les données mais toute tentative de modification est immédiatement bloquée par MongoDB avec un message d'erreur clair.

4.3 Application d'une mesure de sécurité : activation de l'audit

Pour renforcer la sécurité du projet La Table Commune, nous avons choisi d'activer la fonctionnalité d'audit de MongoDB. Cette mesure permet d'enregistrer toutes les opérations importantes réalisées sur la base de données : connexions, modifications de données, changements de rôles, etc.

Pourquoi cette mesure est-elle importante dans notre projet ?

La Table Commune est un système de gestion de restaurant qui traite des données personnelles (noms, coordonnées, historique de commandes) et des données financières (montants des commandes, paiements). En cas d'accès non autorisé ou de modification suspecte, il est crucial de pouvoir retracer exactement qui a fait quoi et quand. Sans audit, il serait impossible d'identifier l'origine d'une faille de sécurité ou d'une modification non désirée.

Étape 1 : Configuration de l'audit dans mongod.cfg

On configure l'audit dans le fichier mongod.cfg de MongoDB. On choisit de journaliser toutes les insertions dans la collection commande, ce qui est le cas le plus critique dans notre projet, ainsi que les connexions et déconnexions.

```
# Dans le fichier mongod.cfg
# Activer l'audit et écrire dans un fichier JSON

auditLog:
  destination: file
  format: JSON
  path: C:\data\logs\audit_LaTableCommune.json
  filter: '{
    "$or": [
      { "atype": "insert", "ns": { "$in":
["LaTableCommune.commande", "LaTableCommune.client"] } },
      { "atype": { "$in": ["authenticate", "logout",
"dropCollection", "createCollection"] } }
    ]
  }'
```

```
# Redémarrer MongoDB après modification
```

Étape 2 : Vérifier que l'audit enregistre bien les opérations

Après avoir redémarré MongoDB avec la nouvelle configuration, on effectue une insertion de test et on vérifie que cette action a bien été tracée dans le fichier d'audit.

```
// Insérer une commande de test
use LaTableCommune

db.commande.insertOne({
  _id: "TEST_AUDIT_001",
  idClient: ObjectId("507f1f77bcf86cd799439011"),
  idSuccursale: ObjectId("507f1f77bcf86cd799439022"),
  dateCommande: new Date(),
  statutCommande: "test",
  montantTotal: 0.01
})

// Vérifier le fichier d'audit (extrait attendu) :
// {
//   "atype": "insert",
//   "ts": { "$date": "2025-04-17T12:00:00.000Z" },
//   "local": { "ip": "127.0.0.1", "port": 27017 },
//   "users": [{ "user": "appUser", "db": "admin" }],
//   "ns": { "db": "LaTableCommune", "coll": "commande" },
//   "result": 0
// }
```

Récapitulatif des bénéfices de cette mesure pour La Table Commune

L'activation de l'audit apporte plusieurs avantages concrets pour notre projet. Premièrement, chaque insertion dans les collections sensibles (commande, client) est tracée avec l'identité de l'utilisateur, la date et l'heure exactes. Deuxièmement, les connexions et déconnexions sont enregistrées, ce qui permet de détecter des accès inhabituels. Troisièmement, les opérations dangereuses comme la suppression de collections sont également capturées, ce qui protège contre les erreurs ou les actes malveillants. Finalement, ces journaux d'audit peuvent être consultés par le responsable informatique du restaurant pour effectuer des vérifications régulières ou répondre à des incidents de sécurité.

7. Références

[1] Notes de cours — *Bases de données massives (NoSQL / MongoDB)* Cours dispensé dans le cadre du programme, session en cours. (Référence à adapter selon le [2] MongoDB, Inc. (2025). *MongoDB Manual — Documentation officielle (v8.2)*. <https://www.mongodb.com/docs/manual/>

[3] MongoDB, Inc. (2025). *Data Modeling in MongoDB*. La modélisation des données dans MongoDB repose sur le principe central que les données accédées ensemble doivent être stockées ensemble, en structurant le modèle selon les patterns d'accès de l'application. <https://www.mongodb.com/docs/manual/data-modeling/>

[4] MongoDB, Inc. (2025). *Best Practices for Data Modeling in MongoDB*. MongoDB recommande de planifier et concevoir le schéma tôt dans le processus de développement, afin de prévenir les problèmes de performance à mesure que l'application grandit. <https://www.mongodb.com/docs/manual/data-modeling/best-practices/>

[5] MongoDB, Inc. (2025). *Schema Design Patterns*. MongoDB propose plusieurs patterns de conception de schéma permettant d'optimiser le modèle de données selon la façon dont l'application interroge et utilise les données, incluant notamment le Computed Pattern et le Polymorphic Pattern. <https://www.mongodb.com/docs/manual/data-modeling/design-patterns/>

[6] MongoDB, Inc. (2025). *Designing Your Schema — Schema Design Process*. Le processus de conception recommandé consiste à identifier la charge de travail, cartographier les relations, appliquer des patterns de conception, puis créer des index pour soutenir les requêtes les plus fréquentes. <https://www.mongodb.com/docs/manual/data-modeling/schema-design-process/>

[7] MongoDB, Inc. (2025). *Indexes — MongoDB Manual*. <https://www.mongodb.com/docs/manual/indexes/>

[8] MongoDB, Inc. (2025). *CRUD Operations — MongoDB Manual*. <https://www.mongodb.com/docs/manual/crud/>

[9] MongoDB, Inc. (2025). *Aggregation Pipelines*. <https://www.mongodb.com/docs/manual/aggregation/>

8. Références

- [1] MongoDB, Inc. (2025). *Aggregation Pipelines*.
<https://www.mongodb.com/docs/manual/aggregation/>
- [2] MongoDB, Inc. (2025). *MongoDB Manual — Documentation officielle (v8.2)*.
<https://www.mongodb.com/docs/manual/>
- [3] MongoDB, Inc. (2025). *Data Modeling in MongoDB*. La modélisation des données dans MongoDB repose sur le principe central que les données accédées ensemble doivent être stockées ensemble, en structurant le modèle selon les patterns d'accès de l'application. <https://www.mongodb.com/docs/manual/data-modeling/>
- [4] MongoDB, Inc. (2025). *Best Practices for Data Modeling in MongoDB*. MongoDB recommande de planifier et concevoir le schéma tôt dans le processus de développement, afin de prévenir les problèmes de performance à mesure que l'application grandit. <https://www.mongodb.com/docs/manual/data-modeling/best-practices/>
- [5] MongoDB, Inc. (2025). *Schema Design Patterns*. MongoDB propose plusieurs patterns de conception de schéma permettant d'optimiser le modèle de données selon la façon dont l'application interroge et utilise les données, incluant notamment le Computed Pattern et le Polymorphic Pattern. <https://www.mongodb.com/docs/manual/data-modeling/design-patterns/>
- [6] MongoDB, Inc. (2025). *Designing Your Schema — Schema Design Process*. Le processus de conception recommandé consiste à identifier la charge de travail, cartographier les relations, appliquer des patterns de conception, puis créer des index pour soutenir les requêtes les plus fréquentes. <https://www.mongodb.com/docs/manual/data-modeling/schema-design-process/>
- [7] MongoDB, Inc. (2025). *Indexes — MongoDB Manual*.
<https://www.mongodb.com/docs/manual/indexes/>
- [8] MongoDB, Inc. (2025). *CRUD Operations — MongoDB Manual*.
<https://www.mongodb.com/docs/manual/crud/>